

Software Quality

Software quality means fitness for purpose: the software fulfills its explicit and implicit requirements and has the necessary quality characteristics, achieved through systematic software engineering and quality assurance—not just testing.

Why do we have to deal with software quality:

Software errors have serious consequences that can be physical or economical, or even scientific like the delay of discovering the ozone layer damage. Modern society relies heavily on software so insuring that the software works properly is absolutely important. Quality cannot be tested into a product, only performing automated tests is not enough to ensure quality, quality is ensured through systematic across the entire development process.

What are the challenges of software quality assurance?:

The main challenges of software quality assurance are that quality is complex and context-dependent, cannot be achieved by testing alone, software is highly complex, resources are limited, and effective QA requires coordinated organizational, constructive, and analytical measures throughout the entire development process.

Quality Management Vs Quality Assurance:

Quality Management: is a framework made to decide on what actually quality is in our software, and how we can achieve a quality software.

Quality Assurance: The rules, tests that we follow in order to achieve the quality in our software. Making sure that the rules we set in our quality management framework is actually being implemented.

Quality Management:

Requirement Engineering: describes what quality is. the quality is described by meeting functional and non functional requirements of a software.

SW Testing: how can the quality be tested, we need methods to test whether the qualities has been met or not. How the quality of the software should be assessed.

SW Testing: how can quality be achieved: by conducting tests on so many levels. (unit testing, integration testing,...) peer reviews.

SW Testing: How to find errors or defects: automated tests, SAST, DAST, Peer code review, monitoring and logging.

SWE practices: How to avoid errors and defects: adhering to code standards and best practices, automated testing pipelines, code reviews.

Quality Assurance:

Organizational (Organization): how the organization can actually work to ensure the quality is met, not in a technical way but in an organizational way (milestones, agile, project management). It focuses on setting up the environment and making sure everyone is following the defined procedures. It also includes training and mentoring.

Constructive (Prevention): how to work in a way that reduces defects and ensures that quality is defined and can be achieved. Following software engineering best practices and design patterns, making tests before actually writing code.

Analytical (Diagnosis): actually testing the software for quality (static testing, peer reviews)

TLDR: o organizational focuses on the environment and making sure everyone follow the agreed upon rules, constructive focuses on making the product of higher quality while writing the code it does not focus on finding errors but rather preventing them, and analytical focuses on just finding the defects

Prevention > Detection:

Organizational and constructive measures are significantly more important, they are supplemented by analytical measures.

- **Prevention > Detection:**
Fixing defects after they occur (analytical QA) is **more expensive and time-consuming** than preventing them (constructive QA).
- **Sustainability:**
Without strong organizational QA, processes are inconsistent, and even preventive measures might fail.
- **Efficiency:**
Fewer defects from the start mean less testing, debugging, and rework later.

Types and examples of Analytical (Diagnosis):

Software Testing: its split into 3 types

- **Reviews:** its done by peer reviews by other developers in the team (manually done by humans)
- **Static:** the computer automatically checks the code for vulnerabilities (Analysis using computer)
- **Dynamic:** the computer executes the code automatically and look for some threats (automatic execution by computer)

Certified Tester Foundation Level (CTFL):

What is it: an entry-level certification in software testing offered by the International Software Testing Qualifications Board (ISTQB®). It's designed to give a solid foundation in the principles and practices of software testing and is the most widely recognized basic qualification in the field worldwide. CTFL has been updated to version 4.0 to align with modern testing practices, including Agile, DevOps, and continuous testing.

The certified tester foundation level is for project managers, agile coaches, beginners and aspiring software testers, developers who want to have more knowledge about the testing process while developing.

Effects of Software Errors:

NASA - Earth observation satellites 1979-1985:

- **What:** NASA satellites were collecting atmospheric data including those of the ozone layer, however, the satellite could not notice the ozone layer hole over antartica for 7 years.
- **Causes:** a software error in data processing and validation, the extremely low ozone levels were treated as outliers and totally ignored by the system software.
- **Effects (damages):** delayed realization of ozone layer damage for 7 years, delay in environmental and political actions done to save the ozone layer.

ESA, Kourou, French Guyana, June 4, 1996:

- **What:** Ariane 5 space rocket was exploded 39 second after its launch this was caused by the safety system blowing the whole thing up mistakenly.
- **Causes:** the software from the past model Ariane 4 was used directly without proper testing. This caused a numerical conversion issue.
- **Effects (damages):** total loss of the rocket of 370 million USD and the reputation of the company.

Manned NASA space capsule Gemini V:

- What: Gemini V, a manned space mission, missed its planned landing site by about 160 kilometers. Astronauts were not harmed, but the mission accuracy was compromised.
- Causes: insufficient validation for the logic and numerical calculation, totally missing the earth rotation around the sun.
- Effects (damages): reduced mission precision. Endangering the safety of the astronaut.

T-Mobile Germany 2009:

- What: during 2009 the network had a problem that caused all the T-mobile network users nationwide to get a “this number is not available”
- Causes: a software problem with the home location register (HLR)
- Effects (damages): many businesses and humans were hurt due to the absence of communication. The mobile network lost much reputation. Customer compensations causing the company to lose much money.

Other examples:

Audi airbag failure

T-Mobile Outage

Cause:

- A backend software error caused network congestion.
- Backend systems handle call routing, subscriber management, and traffic control.
- When the software malfunctioned, the network could not properly manage traffic loads.

Effect:

- Nationwide outage (USA).
- Customers experienced dropped calls, unreachable phones, and unavailable services.
- Business operations, emergency communication, and customer trust were affected.

CrowdStrike Error

Cause:

- A faulty kernel module was deployed.
- Kernel modules operate at the lowest and most privileged level of an operating system.
- The faulty update caused critical system crashes during startup.

Effect:

- Millions of Windows systems became unbootable.
- Affected organizations included airlines, hospitals, banks, and businesses worldwide.
- Systems required manual recovery, leading to massive operational disruptions.

Verizon Billing Error

Cause:

- A programming error in the billing system.
- Incorrect logic led to customers being charged for plans or services they did not use or agree to.

Effect:

- Billing plans were inadvertently charged.
- Customer dissatisfaction and complaints increased.
- Verizon faced administrative costs, refunds, and potential regulatory scrutiny.

GM Brake Warning Failure

Cause:

- An unspecified software error in the vehicle's control or monitoring system.
- The system failed to correctly detect or signal low brake fluid levels.

Effect:

- Brake fluid warning light failed to turn on.
- Drivers were not alerted to a potentially dangerous condition.

What does this mean for me:

software errors can cause massive financial, operational, and safety damage, meaning that I must treat software quality as a professional responsibility and apply organizational, constructive, and analytical quality assurance consistently.

Testing alone is not enough! We should follow procedures, we should take constructive quality assurance, and organizational into account.

What do you do in the area of quality management and testing in industry?:

Development: the more you understand about testing the better you are at development. Building quality into the code make it easier for the quality to be achieved. It makes it easier for the developer to solve the problems found by the quality assurance team.

Process Management: ensure that development and testing process are defined and followed. Define development and QA procedures, make sure that everything is documented and traceable.

Methods/ Tool teams: Provide methods, guidelines, and tools that help teams build quality. Like tools for the DevOps team for the CI/CD

Testing: Find issues before they reach costumers, QA analytical type.

Test Management/ release Management/ Software Approval: decide when the software is ready to be released.

Test Data Management: Consturctive and analytical QA, makes sure that tests are done and complete and realistic.

Special Disciplines (advanced test area):